

# Architettura BLE lato Client e Server

Andrea Zorzi – BillyApp

Versione 1.1.1  
25 novembre 2025

## Indice

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Scopo e perimetro</b>                            | <b>3</b> |
| <b>2</b> | <b>Panoramica generale</b>                          | <b>3</b> |
| 2.1      | Attori Principali . . . . .                         | 3        |
| 2.1.1    | Client trasmittente . . . . .                       | 3        |
| 2.1.2    | Client ricevente . . . . .                          | 3        |
| 2.1.3    | API e database . . . . .                            | 3        |
| 2.2      | Principio di base . . . . .                         | 3        |
| 2.3      | Ruolo del client . . . . .                          | 3        |
| 2.3.1    | Advertising BLE . . . . .                           | 4        |
| 2.3.2    | Scanning BLE . . . . .                              | 4        |
| 2.3.3    | Risoluzione tramite backend . . . . .               | 4        |
| 2.4      | Flusso logico . . . . .                             | 4        |
| 2.4.1    | Generazione degli identificativi $B_{id}$ . . . . . | 4        |
| 2.4.2    | Trasmissione via BLE . . . . .                      | 4        |
| 2.4.3    | Raccolta dei $B_{id}$ . . . . .                     | 4        |
| 2.4.4    | Risoluzione lato backend . . . . .                  | 4        |
| 2.4.5    | Presentazione sull'APP . . . . .                    | 4        |
| <b>3</b> | <b>Requisiti di progetto</b>                        | <b>4</b> |
| 3.1      | Privacy . . . . .                                   | 5        |
| 3.2      | Non tracciabilità . . . . .                         | 5        |
| 3.3      | Efficienza BLE . . . . .                            | 5        |
| 3.4      | Scalabilità . . . . .                               | 5        |
| <b>4</b> | <b>Modello dati dell'utente</b>                     | <b>5</b> |
| <b>5</b> | <b>Design crittografico</b>                         | <b>5</b> |
| 5.1      | Obiettivo della cifratura . . . . .                 | 5        |
| 5.2      | Finestra temporale . . . . .                        | 6        |
| 5.3      | Chiavi AES e rotazione . . . . .                    | 6        |
| 5.3.1    | Politica di rotazione . . . . .                     | 7        |
| 5.3.2    | Effetti di sicurezza . . . . .                      | 8        |
| <b>6</b> | <b>Flusso del client</b>                            | <b>8</b> |
| 6.1      | Gestione advertising BLE . . . . .                  | 8        |
| 6.2      | Gestione scanning BLE . . . . .                     | 9        |
| 6.3      | Gestione batch e condizioni di errore . . . . .     | 9        |

|           |                                                                      |           |
|-----------|----------------------------------------------------------------------|-----------|
| 6.3.1     | Primo avvio o nuovo login . . . . .                                  | 10        |
| 6.3.2     | Batch scaduto . . . . .                                              | 10        |
| <b>7</b>  | <b>Flusso di risoluzione lato server</b>                             | <b>10</b> |
| 7.1       | Raccolta del $B_{id}$ dal client . . . . .                           | 10        |
| 7.2       | Decifratura, selezione chiave e condizione di accettazione . . . . . | 10        |
| 7.2.1     | Algoritmo di risoluzione per un singolo $B_{id}$ . . . . .           | 11        |
| 7.2.2     | Risposta al client . . . . .                                         | 13        |
| <b>8</b>  | <b>Threat model e mitigazioni</b>                                    | <b>13</b> |
| 8.1       | Sniffing BLE passivo . . . . .                                       | 13        |
| 8.2       | Tracciamento nel tempo . . . . .                                     | 13        |
| 8.3       | Replay e spoofing di $B_{id}$ . . . . .                              | 14        |
| 8.4       | Compromissione di chiavi o sistemi backend . . . . .                 | 14        |
| 8.5       | Compromissione o furto del dispositivo utente . . . . .              | 14        |
| <b>9</b>  | <b>Alternative considerate e modelli mitigati</b>                    | <b>15</b> |
| 9.1       | Nome in chiaro via BLE . . . . .                                     | 15        |
| 9.2       | Identificativo statico non rotante . . . . .                         | 15        |
| 9.3       | Identificativi random rotanti con mapping esplicito . . . . .        | 16        |
| 9.4       | Modello HMAC-SHA-256 con tabelle di lookup . . . . .                 | 16        |
| 9.5       | Modello AES server-centrico precedente . . . . .                     | 17        |
| <b>10</b> | <b>Sintesi complessiva</b>                                           | <b>18</b> |
| 10.1      | Architettura in sintesi . . . . .                                    | 18        |
| 10.2      | Proprietà di sicurezza e privacy . . . . .                           | 20        |
| 10.3      | Scalabilità e complessità . . . . .                                  | 20        |
| 10.4      | Trade-off e limiti noti . . . . .                                    | 20        |

# 1 Scopo e perimetro

Questo documento descrive l'architettura logica e tecnica del sistema BLE lato client e lato server

L'idea è avere un'unica base di riferimento che permetta di:

- capire come il client genera, ruota e trasmette l'identificativo BLE  $B_{id}$ ;
- capire come il backend risolve  $B_{id}$  in un utente reale;
- avere una vista completa sulle scelte di privacy e sicurezza;
- alternative valutate e perché sono state scartate o mitigate.

Il perimetro è architettuale, non di implementazione. Non sono quindi presenti dettagli di codice, ma solo di logica e flussi. L'implementazione sarà trattata in un documento a parte.

## 2 Panoramica generale

### 2.1 Attori Principali

Il sistema coinvolge i seguenti attori principali:

#### 2.1.1 Client trasmittente

Il dispositivo del client trasmette periodicamente via BLE un identificativo cifrato  $B_{id}$ . Questo valore non contiene informazioni personali o altri dati personali in chiaro.

#### 2.1.2 Client ricevente

Il dispositivo del client ricevente ascolta i pacchetti BLE nell'area, li filtra per il nostro servizio, raccoglie i  $B_{id}$  e li invia al backend tramite una richiesta autenticata. In cambio riceve i dettagli delle persone vicine.

#### 2.1.3 API e database

Il backend riceve i  $B_{id}$  dai client riceventi, li risolve in utenti reali e restituisce i dettagli delle persone vicine.

- conosce tutti gli utenti
- gestisce le chiavi crittografiche
- genera i pacchetti BLE  $B_{id}$  per i client trasmittenti
- risolve i  $B_{id}$  per i client riceventi
- comunica con il client ricevente tramite API le informazioni necessarie

### 2.2 Principio di base

Via radio BLE passa soltanto l'identificativo cifrato  $B_{id}$ . Tutto ciò che il client ricevente conosce sugli utenti vicini lo apprende soltanto tramite il backend via API, mai in chiaro su BLE. In questo modo il canale radio resta molto povero di informazioni ed i dati sensibili rimangono confinati al backend.

### 2.3 Ruolo del client

L'app svolge tre funzioni principali:

### 2.3.1 Advertising BLE

Genera e trasmette periodicamente nei pacchetti BLE l'identificativo cifrato  $B_{id}$  valido per lo slot temporale corrente. Ogni intervallo di tempo viene utilizzato un nuovo  $B_{id}$ . Il client ha con sè in memoria i  $B_{id}$  generati per le successive 24 ore, che vengono mandati dal backend in un batch.

### 2.3.2 Scanning BLE

Ascolta i pacchetti BLE nell'area circostante, filtra quelli rilevanti per il nostro servizio e raccoglie i  $B_{id}$  visti.

### 2.3.3 Risoluzione tramite backend

Invia una richiesta al backend con i  $B_{id}$  raccolti, riceve in risposta i dettagli delle persone vicine e li mostra all'utente.<sup>1</sup>

## 2.4 Flusso logico

Il flusso complessivo è il seguente:

### 2.4.1 Generazione degli identificativi $B_{id}$

Per ogni utente e per ogni intervallo di tempo il backend genera un valore cifrato  $B_{id}$  usando una cifratura AES-ENC ed una chiave master AES-256. Il backend genera i successivi 144  $B_{id}$  e li manda al client in un batch.

### 2.4.2 Trasmissione via BLE

Il client inserisce il  $B_{id}$  valido per lo slot temporale corrente nei pacchetti BLE e li trasmette periodicamente.

### 2.4.3 Raccolta dei $B_{id}$

Il client ricevente ascolta i pacchetti BLE nell'area, estrae i  $B_{id}$  e lo invia al backend per ottenere le informazioni dell'utente vicino.

### 2.4.4 Risoluzione lato backend

Il backend riceve i  $B_{id}$  dai client riceventi, e li decifra usando la chiave master AES-256 corretta. Dai  $B_{id}$  decifrati ottiene  $U_{code}$  ed il time slot. Applica i controlli di validità temporale e restituisce i dettagli degli utenti vicini al client ricevente.

### 2.4.5 Presentazione sull'APP

Nell'app del client ricevente vengono mostrati i dettagli delle persone vicine ricevuti dal backend.

## 3 Requisiti di progetto

I requisiti principali del sistema sono:

---

<sup>1</sup>Sul dispositivo non esiste un mapping persistente  $B_{id} \rightarrow$  utente. La risoluzione di informazioni delle persone è sempre centralizzata nel backend.

### 3.1 Privacy

- Nessun dato personale o sensibile deve essere trasmesso in chiaro via BLE;
- Un osservatore esterno che sniffa i pacchetti BLE non deve poter risalire in modo affidabile all'identità di un utente;
- Il mapping  $B_{id} \rightarrow$  utente deve essere noto solo al backend.

### 3.2 Non tracciabilità

- Un osservatore esterno che sniffa i pacchetti BLE non deve poter seguire gli spostamenti di un utente nel tempo basandosi sull'identificativo  $B_{id}$ ;
- Anche disponendo di log storici dei  $B_{id}$  trasmessi non deve essere possibile risalire agli spostamenti di un utente.

### 3.3 Efficienza BLE

- I pacchetti BLE devono essere il più piccoli possibile per ridurre il consumo energetico;
- Deve semplificare l'implementazione lato client tra gli stack BLE di iOS e Android.

### 3.4 Scalabilità

- L'architettura deve poter supportare milioni di utenti attivi;
- La risoluzione di un singolo  $B_{id}$  deve essere  $\approx O(1)$ , ovvero efficiente e non dipendente dal numero di utenti totali.
- Il backend deve poter gestire migliaia di richieste al secondo.
- Il client deve poter gestire centinaia di  $B_{id}$  raccolti.

## 4 Modello dati dell'utente

**Definizione ( $U_{code}$ ):** Sia

$$U_{code} = 64 \text{ bit} = 8 \text{ B}$$

un identificativo univoco per ogni utente, generato casualmente al momento della registrazione. Offre un namespace di  $2^{64} \approx 1.84 \times 10^{19}$  utenti, sufficiente per l'applicazione. Non viene mai trasmesso in chiaro via BLE.

Ogni utente nel sistema è rappresentato dai seguenti dati:

- identificativo univoco dell'utente  $U_{code}$ ;
- nome visualizzato;

Questi dati vengono usati solo nella comunicazione tra client ricevente e backend.

## 5 Design crittografico

### 5.1 Obiettivo della cifratura

L'obiettivo principale della cifratura è:

- semplicità e efficienza di una soluzione AES con crittografia reversibile;

- autonomia del client nella rotazione degli identificativi  $B_{id}$  senza dover contattare il backend frequentemente;

## 5.2 Finestra temporale

Il tempo viene discretizzato in slot regolari di durata fissa.

**Definizione ( $\Delta t_{slot}$ ):** Sia

$$\Delta t_{slot} = 600 \text{ s} = 10 \text{ min}$$

la durata di uno slot temporale.

**Definizione ( $t_{unix}$ ):** Sia

$$t_{unix} \in \mathbb{N}, \quad [t_{unix}] = \text{s},$$

il tempo corrente in secondi trascorsi dal 1 gennaio 1970.

**Definizione ( $S_{slot}$ ):** Sia

$$S_{slot} = \left\lfloor \frac{t_{unix}}{\Delta t_{slot}} \right\rfloor \in \mathbb{N}$$

l'indice dello slot temporale corrente.

$S_{slot}$  è concettualmente pubblico e non segreto, in quanto può essere calcolato da chiunque conoscendo l'ora corrente.

Quando un identificativo  $B_{id}$  viene decifrato nel backend, si ottiene in particolare un indice di slot codificato

$$S_{slot}^{dec} \in \mathbb{N},$$

che rappresenta lo slot temporale a cui il  $B_{id}$  è destinato.

**Definizione ( $\delta$ ):** Sia

$$\delta = \frac{\Delta t_{slot}}{5}$$

la tolleranza temporale simmetrica per la risoluzione dei  $B_{id}$ . Il suo valore è quindi dinamicamente legato alla durata dello slot: se  $\Delta t_{slot}$  cambia, cambia proporzionalmente anche  $\delta$ .

Dallo slot decifrato si ricavano i tempi di inizio e fine dello slot codificato:

$$t_{start}^{dec} = S_{slot}^{dec} \cdot \Delta t_{slot}, \quad t_{end}^{dec} = (S_{slot}^{dec} + 1) \cdot \Delta t_{slot}.$$

Un identificativo  $B_{id}$  ricevuto al tempo  $t_{unix}$  è temporalmente valido se e solo se

$$t_{start}^{dec} - \delta \leq t_{unix} \leq t_{end}^{dec} + \delta.$$

In altre parole, il backend accetta un  $B_{id}$  se il tempo corrente cade all'interno dello slot codificato, esteso di una tolleranza  $\delta$  prima dell'inizio e  $\delta$  dopo la fine. Questa finestra allargata assorbe il drift di orologio tra client e backend e la latenza di rete, senza allargare troppo l'esposizione temporale di un singolo  $B_{id}$ .

## 5.3 Chiavi AES e rotazione

La generazione dei  $B_{id}$  si basa su chiavi master AES-256. Le chiavi non sono statiche e vengono ruotate periodicamente.

**Definizione ( $K_i$ ):** Sia

$$K_i \in \{0, 1\}^{256}, \quad |K_i| = 256 \text{ bit} = 32 \text{ byte}, \quad \forall i \in \mathbb{N}$$

una chiave master AES-256 usata per generare i  $B_{\text{id}}$ . Ogni  $K_i$  è generata tramite CSPRNG e memorizzata in modo sicuro nel backend. Le chiavi non derivano da una root comune, ma sono valori casuali forti e indipendenti. A ciascun  $K_i$  è associato un intervallo di tempo.

**Definizione** ( $T_{K_i}, T_{K_i, \text{grazia}}$ ): Sia

$$T_{K_i} = 72 \text{ h}$$

la durata di validità della chiave master  $K_i$ . Rispettivamente, sia

$$T_{K_i, \text{grazia}} = \frac{T_{K_i}}{5}$$

la durata del periodo di grazia successiva alla scadenza di  $K_i$ , durante la quale  $K_i$  è ancora accettata per la risoluzione dei  $B_{\text{id}}$ .

**Definizione** ( $(K_i)_{i \in \mathbb{N}}$ ): Sia

$$(K_i)_{i \in \mathbb{N}}$$

la sequenza di chiavi master AES-256 usate per generare i  $B_{\text{id}}$ . Ogni chiave è valida per un intervallo di tempo fisso.

**Definizione** ( $t_{K_i}^{\text{start}}, t_{K_i}^{\text{end}}, t_{K_i}^{\text{grazia}}$ ): Sia

$$t_{K_i}^{\text{start}} = \begin{cases} t_{\text{now}}, & i = 1, \\ t_{K_{i-1}}^{\text{end}}, & i > 1, \end{cases} \quad i \in \mathbb{N}, i \geq 1.$$

il tempo di inizio validità di  $K_i$ . Sia inoltre

$$t_{K_i}^{\text{end}} = t_{K_i}^{\text{start}} + T_{K_i}, \quad \forall i \in \mathbb{N}.$$

il tempo di fine validità di  $K_i$ . Infine, sia

$$t_{K_i}^{\text{grazia}} = t_{K_i}^{\text{end}} + T_{K_i, \text{grazia}}, \quad \forall i \in \mathbb{N}.$$

il tempo di grazia, dove  $K_i$  non è più usata per generare nuovi  $B_{\text{id}}$ , ma è ancora accettata ed utilizzata per la risoluzione.

### 5.3.1 Politica di rotazione

La rotazione delle chiavi è periodica, e l'intervallo tra l'attivazione di chiavi consecutive è costante

$$T_{K_i} = t_{K_i}^{\text{start}} - t_{K_{i-1}}^{\text{start}}, \quad \forall i > 1.$$

$K_i$  è la chiave corrente nel tempo  $t_{\text{now}}$  se e solo se:

$$t_{K_i}^{\text{start}} \leq t_{\text{now}} < t_{K_i}^{\text{end}}$$

Mentre  $K_{i-1}$  è la chiave precedente ancora accettata nel periodo di grazia se e solo se:

$$t_{K_{i-1}}^{\text{end}} = t_{K_i}^{\text{start}} < t_{\text{now}} \leq t_{K_{i-1}}^{\text{grazia}}$$

La politica di rotazione è progettata in modo che, per ogni  $t_{\text{now}}$ ,

$$\forall t \in \mathbb{R}, \quad \left| \left\{ i \in \mathbb{N} \mid t_{K_i}^{\text{start}} \leq t \leq t_{K_i}^{\text{grazia}} \right\} \right| \leq 2.$$

### 5.3.2 Effetti di sicurezza

Questa architettura garantisce che:

- l'impatto temporale di una eventuale compromissione di chiave è limitato a  $T_{K_i} + T_{K_i, \text{grazia}}$ ;
- un attaccante che compromette  $K_i$  non può risalire ai  $B_{\text{id}}$  generati con chiavi precedenti  $K_{i-1}$ ,  $i - 1 < i$ ;
- la rotazione delle chiavi  $K_i$ , il periodo di grazia  $T_{K_i, \text{grazia}}$  e la tolleranza temporale  $\delta$  sono gestiti in modo centralizzato e configurabile solo nel backend;
- tutto il materiale crittografico resta confinato nel backend.

## 6 Flusso del client

Descriviamo in modo più formale cosa fa il client, sia quando trasmette sia quando ascolta i pacchetti BLE  $B_{\text{id}}$

**Definizione ( $T_{\text{batch}}$ ):** Sia

$$T_{\text{batch}} = 24 \text{ h}$$

la durata di validità di un batch di  $B_{\text{id}}$  inviato dal backend al client trasmittente. Rappresenta l'intervallo di tempo in cui il client è in grado di generare i  $B_{\text{id}}$  localmente senza dover contattare il backend (offline).

**Definizione ( $N_{\text{batch}}$ ):** Sia

$$N_{\text{batch}} = \frac{T_{\text{batch}}}{\Delta t_{\text{slot}}}$$

il numero di  $B_{\text{id}}$  contenuti in un batch inviato dal backend al client trasmittente.

### 6.1 Gestione advertising BLE

**Definizione ( $(B_{\text{id},k})_{k=1}^{N_{\text{batch}}}$ ):** Sia

$$\mathbf{B} = (B_{\text{id},k})_{k=1}^{N_{\text{batch}}}, \quad B_{\text{id},k} \in \{0, 1\}^{128}, \quad \forall k \in \{1, \dots, N_{\text{batch}}\}.$$

una sequenza ordinata di identificativi  $B_{\text{id}}$

Il client trasmittente mantiene in memoria locale un batch di identificativi  $B_{\text{id}}$  fornito dal backend, valida per  $N_{\text{batch}}$  di  $S_{\text{slot}}$  consecutivi.

**Definizione ( $S_j$ ):** Sia l'indice del primo slot del batch  $S_1 \in \mathbb{N}$  con

$$S_j = S_1 + (j - 1), \quad \forall j \in \{1, \dots, N_{\text{batch}}\},$$

l'indice dello slot temporale coperto dal batch a cui è destinato l'identificativo  $B_{\text{id},j}$ .

Nell'istante di advertising BLE, il client calcola lo slot corrente

$$S_{\text{slot}}^{\text{curr}} = \left\lfloor \frac{t_{\text{unix}}}{\Delta t_{\text{slot}}} \right\rfloor.$$

$$j^* = S_{\text{slot}}^{\text{curr}} - S_1 + 1,$$

ed usa l'identificativo  $B_{\text{id},j^*}$  da mandare a condizione che

$$1 \leq j^* \leq N_{\text{batch}}.$$

Il ciclo operativo si può riassumere così:



1. Il backend genera un batch di identificativi

$$\mathbf{B} = (B_{\text{id},j})_{j=1}^{N_{\text{batch}}}$$

e l'indice del primo slot del batch  $S_1$ , e li invia al client trasmittente.

2. Il client memorizza localmente  $\mathbf{B}$  e  $S_1$ , validi per  $N_{\text{batch}}$  slot temporali consecutivi.
3. Ogni volta che deve trasmettere un pacchetto BLE, il client misura il tempo corrente  $t_{\text{unix}}$  e calcola

$$S_{\text{slot}}^{\text{curr}} = \left\lfloor \frac{t_{\text{unix}}}{\Delta t_{\text{slot}}} \right\rfloor.$$

4. Dal valore di  $S_{\text{slot}}^{\text{curr}}$  ricava l'indice nel batch

$$j^* = S_{\text{slot}}^{\text{curr}} - S_1 + 1.$$

5. Se

$$1 \leq j^* \leq N_{\text{batch}},$$

il client inserisce  $B_{\text{id},j^*}$  nel pacchetto BLE e lo trasmette.

6. Se invece  $j^* < 1$  oppure  $j^* > N_{\text{batch}}$ , il batch corrente non copre più lo slot attuale e il client deve ottenere un nuovo batch dal backend eliminando quello attuale.

## 6.2 Gestione scanning BLE

Il client ricevente si occupa di ascoltare e registrare i  $B_{\text{id}}$  degli altri dispositivi. Per ogni istante La logica, per ogni pacchetto BLE ricevuto, è:

1. filtrare solo i pacchetti BLE del nostro servizio ed ignorare gli altri;
2. estrarre il  $B_{\text{id}}$  dal pacchetto;
3. inviare al backend la richiesta di risoluzione del pacchetto  $B_{\text{id}}$

Ogni  $B_{\text{id}}$  viene quindi risolto al massimo una volta.

## 6.3 Gestione batch e condizioni di errore

Il client deve gestire alcuni casi particolari legati alla disponibilità di un batch valido e alla risposta del backend.

Un batch

$$\mathbf{B} = (B_{\text{id},j})_{j=1}^{N_{\text{batch}}}, \quad B_{\text{id},j} \in \{0, 1\}^{128},$$

con primo slot  $S_1 \in \mathbb{N}$  è *valido* all'istante  $t_{\text{unix}}$  se e solo se

$$S_{\text{slot}}^{\text{curr}} = \left\lfloor \frac{t_{\text{unix}}}{\Delta t_{\text{slot}}} \right\rfloor$$

soddisfa

$$S_1 \leq S_{\text{slot}}^{\text{curr}} \leq S_1 + N_{\text{batch}} - 1.$$

Equivalentemente, esiste un indice

$$j^* = S_{\text{slot}}^{\text{curr}} - S_1 + 1$$

tale che

$$1 \leq j^* \leq N_{\text{batch}},$$

e l'identificativo da usare è  $B_{\text{id},j^*}$ .

### 6.3.1 Primo avvio o nuovo login

Al primo avvio il client non dispone di un batch valido, richiede immediatamente al backend un nuovo batch che copre le prossime  $T_{\text{batch}}$

### 6.3.2 Batch scaduto

Se il batch corrente è scaduto, cioè

$$S_{\text{slot}}^{\text{curr}} > S_1 + N_{\text{batch}} - 1$$

ed il client non riesce ad ottenere un nuovo batch dal backend (assenza di connettività o errore), allora:

- il client sospende l'advertising BLE;
- riprenderà l'advertising solo dopo aver ricevuto un nuovo batch valido, per evitare di trasmettere  $B_{\text{id}}$  non più risolvibili dal backend

In condizioni normali, finché esiste un batch valido e l'orologio del dispositivo resta sufficientemente sincronizzato, l'app può operare in modo autonomo per una durata pari a  $T_{\text{batch}}$ , riallineandosi al backend non appena necessario.

## 7 Flusso di risoluzione lato server

### 7.1 Raccolta del $B_{\text{id}}$ dal client

Quando il client ha raccolto un  $B_{\text{id}}$ , invia una richiesta al backend. Formalmente, una richiesta tipica contiene:

- il pacchetto BLE  $B_{\text{id}}$  da risolvere

Una singola chiamata permette di risolvere un solo  $B_{\text{id}}$

### 7.2 Decifratura, selezione chiave e condizione di accettazione

**Definizione ( $K_{\text{curr}}(t_{\text{unix}})$ ):** Per ogni istante  $t_{\text{unix}}$  esiste al più un indice  $i \in \mathbb{N}$  tale che

$$t_{K_i}^{\text{start}} \leq t_{\text{unix}} < t_{K_i}^{\text{end}}.$$

Quando tale indice esiste, lo denotiamo con  $i^*$  e definiamo

$$K_{\text{curr}}(t_{\text{unix}}) := K_{i^*}.$$

**Definizione ( $K_{\text{prev}}(t_{\text{unix}})$ ):** Dato  $i^*$  come sopra, se  $i^* > 1$  e

$$t_{K_{i^*-1}}^{\text{end}} = t_{K_{i^*}}^{\text{start}} \quad \text{e} \quad t_{K_{i^*}}^{\text{start}} < t_{\text{unix}} \leq t_{K_{i^*-1}}^{\text{grazia}},$$

allora definiamo

$$K_{\text{prev}}(t_{\text{unix}}) := K_{i^*-1}.$$

**Definizione ( $\mathcal{K}_{\text{cand}}(t_{\text{unix}})$ ):** Sia

$$\mathcal{K}_{\text{cand}}(t_{\text{unix}}) = \left\{ K_i \mid i \in \mathbb{N}, t_{K_i}^{\text{start}} \leq t_{\text{unix}} \leq t_{K_i}^{\text{grazia}} \right\}.$$

l'insieme delle chiavi candidate al tempo  $t_{\text{unix}}$

Quindi le chiavi a disposizione per il backend sono:

$$\mathcal{K}_{\text{cand}}(t_{\text{unix}}) = \begin{cases} \{K_{\text{curr}}(t_{\text{unix}})\}, & \text{se } K_{\text{prev}}(t_{\text{unix}}) \text{ non è definita,} \\ \{K_{\text{curr}}(t_{\text{unix}}), K_{\text{prev}}(t_{\text{unix}})\}, & \text{altrimenti.} \end{cases}$$

Riassumendo il backend dispone di:

- la chiave corrente sempre utilizzabile  $K_{\text{curr}}(t_{\text{unix}})$ ;
- la chiave precedente, utilizzabile se ancora nel suo periodo di grazia  $K_{\text{prev}}(t_{\text{unix}})$ .

Ogni  $B_{\text{id}}$  è il risultato della cifratura AES-ENC di una coppia  $(U_{\text{code}}, S_{\text{slot}}^{(64)})$ , dove  $S_{\text{slot}}^{(64)}$  è la rappresentazione a 64 bit dell'indice di slot temporale.

Dal valore  $S_{\text{slot}}^{(64)}$  decifrato il backend ricava l'indice di slot

$$S_{\text{slot}}^{\text{dec}} \in \mathbb{N}.$$

Al tempo di risoluzione  $t_{\text{unix}}$ , il backend calcola lo slot corrente

$$S_{\text{slot}}^{\text{curr}} = \left\lfloor \frac{t_{\text{unix}}}{\Delta t_{\text{slot}}} \right\rfloor.$$

### 7.2.1 Algoritmo di risoluzione per un singolo $B_{\text{id}}$

Dato un  $B_{\text{id}}$  ricevuto in un istante  $t_{\text{unix}}$ , il backend esegue i passi seguenti.

**Scelta delle chiavi da provare** In base all'istante corrente  $t_{\text{unix}}$ , il backend determina l'insieme delle chiavi candidate

$$\mathcal{K}_{\text{cand}}(t_{\text{unix}}) = \left\{ K_i \mid i \in \mathbb{N}, t_{K_i}^{\text{start}} \leq t_{\text{unix}} \leq t_{K_i}^{\text{grazia}} \right\}.$$

Per costruzione della politica di rotazione, vale

$$1 \leq |\mathcal{K}_{\text{cand}}(t_{\text{unix}})| \leq 2,$$

e le chiavi vengono considerate in ordine di priorità:

$$K_{\text{curr}}(t_{\text{unix}}) \quad \text{prima,} \quad K_{\text{prev}}(t_{\text{unix}}) \quad \text{poi (se definita).}$$

**Tentativi di decifratura** Per ciascuna chiave  $K \in \mathcal{K}_{\text{cand}}(t_{\text{unix}})$ , il backend tenta la decifratura

$$(U_{\text{code}}^{(K)}, S_{\text{slot}}^{(64),(K)}) = \text{AES-DEC}_K(B_{\text{id}}),$$

dove  $U_{\text{code}}^{(K)} \in \{0, 1\}^{64}$  e  $S_{\text{slot}}^{(64),(K)} \in \{0, 1\}^{64}$ .

Se la decifratura fallisce (blocco non valido, padding errato o formato non coerente), la chiave  $K$  viene scartata per quel  $B_{\text{id}}$ . Tutte le coppie  $(U_{\text{code}}^{(K)}, S_{\text{slot}}^{(64),(K)})$  ottenute da decifrate riuscite vengono trattate come *candidate* e passano ai passi di verifica successivi.

Se nessuna chiave in  $\mathcal{K}_{\text{cand}}(t_{\text{unix}})$  produce una decifratura valida, il  $B_{\text{id}}$  viene scartato come invalido.

**Verifica della finestra temporale** Da ciascun valore  $S_{\text{slot}}^{(64),(K)}$  decifrato, il backend ricava l'indice di slot

$$S_{\text{slot}}^{\text{dec},(K)} \in \mathbb{N}.$$

Dallo slot decifrato costruisce i tempi di inizio e fine dello slot codificato:

$$t_{\text{start}}^{\text{dec},(K)} = S_{\text{slot}}^{\text{dec},(K)} \cdot \Delta t_{\text{slot}}, \quad t_{\text{end}}^{\text{dec},(K)} = (S_{\text{slot}}^{\text{dec},(K)} + 1) \cdot \Delta t_{\text{slot}}.$$

All'istante di risoluzione  $t_{\text{unix}}$ , la coppia decifrata con chiave  $K$  è temporalmente valida se e solo se

$$t_{\text{start}}^{\text{dec},(K)} - \delta \leq t_{\text{unix}} \leq t_{\text{end}}^{\text{dec},(K)} + \delta.$$

Solo le coppie  $(U_{\text{code}}^{(K)}, S_{\text{slot}}^{\text{dec},(K)})$  che soddisfano la condizione di finestra temporale vengono considerate per la fase di verifica utente. Se nessuna delle decifrate valide soddisfa la finestra temporale, il backend rifiuta il  $B_{\text{id}}$  come scaduto o troppo distante nel tempo.

**Verifica utente** Per ciascuna coppia temporalmente valida  $(U_{\text{code}}^{(K)}, S_{\text{slot}}^{\text{dec},(K)})$ , il backend esegue un lookup sul proprio database per verificare l'esistenza di un utente associato a  $U_{\text{code}}^{(K)}$ .

In forma concettuale, il backend cerca  $U_{\text{code}}^{(K)}$  tale che  $U_{\text{code}}^{(K)}$  corrisponda a un utente registrato;

**Risultato** Se esiste esattamente una decifratura che supera sia la verifica temporale sia la verifica utente, allora il  $B_{\text{id}}$  viene considerato risolto e il backend restituisce al client ricevente i dettagli dell'utente associato.

Se nessuna decifratura supera entrambe le verifiche, il  $B_{\text{id}}$  viene rifiutato. Per costruzione del modello (spazio delle chiavi e disegno del formato), la probabilità che più decifrate diverse producano coppie coerenti e riferite a utenti validi è:

$$\Pr[\text{collisione di decifratura valida}] \leq \varepsilon, \quad \varepsilon \approx 2^{-128}.$$

**Sintesi** In forma compatta, un  $B_{\text{id}}$  ricevuto al tempo  $t_{\text{unix}}$  viene accettato se e solo se esiste almeno una chiave  $K \in \mathcal{K}_{\text{cand}}(t_{\text{unix}})$  tale che

$$\exists U_{\text{code}} \in \{0, 1\}^{64}, S_{\text{slot}}^{\text{dec}} \in \mathbb{N} \text{ tali che}$$

$$\text{AES-DEC}_K(B_{\text{id}}) = (U_{\text{code}}, S_{\text{slot}}^{(64)}),$$

da cui si ottiene

$$S_{\text{slot}}^{\text{dec}} \in \mathbb{N}, \quad t_{\text{start}}^{\text{dec}} = S_{\text{slot}}^{\text{dec}} \cdot \Delta t_{\text{slot}}, \quad t_{\text{end}}^{\text{dec}} = (S_{\text{slot}}^{\text{dec}} + 1) \cdot \Delta t_{\text{slot}}.$$

La condizione temporale richiede che

$$t_{\text{start}}^{\text{dec}} - \delta \leq t_{\text{unix}} \leq t_{\text{end}}^{\text{dec}} + \delta,$$

e che  $U_{\text{code}}$  identifichi un utente valido/attivo nel backend al tempo  $t_{\text{unix}}$ .

Se nessuna chiave candidata soddisfa tutte le condizioni sopra, il  $B_{\text{id}}$  viene rifiutato.

### 7.2.2 Risposta al client

Per ogni  $B_{\text{id}}$  decifrato, il backend restituisce un risultato sintetico, senza esporre dettagli interni sulla causa di eventuali rifiuti. In generale:

- se tutti i controlli sono superati, il backend restituisce:  
un nome visualizzato
- se uno o più controlli falliscono, il backend restituisce una risposta neutra o un errore generico, senza fornire indizi aggiuntivi.

L'app lato client usa solo queste risposte aggregate per comporre gli utenti vicino a lui, senza mai accedere direttamente ai dettagli del processo di risoluzione interno al backend.

## 8 Threat model e mitigazioni

### 8.1 Sniffing BLE passivo

**Rischio** Un attaccante ascolta passivamente il traffico BLE in un'area e prova a ricostruire quali utenti sono presenti o a costruire uno storico dei loro passaggi nel tempo.

#### Mitigazioni

- i pacchetti BLE contengono solo  $B_{\text{id}} \in \{0, 1\}^{128}$ , dove

$$B_{\text{id}} = \text{AES-ENC}_{K_i}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)}),$$

senza nomi,  $U_{\text{code}}$  in chiaro o altri identificativi stabili;

- per uno stesso utente, il valore  $B_{\text{id}}$  cambia a ogni slot temporale  $\Delta t_{\text{slot}}$ , quindi la sequenza  $(B_{\text{id}}(S_{\text{slot}}))_{S_{\text{slot}}}$  è una famiglia di valori pseudocasuali dipendenti da  $S_{\text{slot}}$ ;
- il mapping

$$B_{\text{id}} \longrightarrow (U_{\text{code}}, S_{\text{slot}})$$

è noto solo al backend e richiede una chiave  $K_i$  valida: per un osservatore che vede solo il traffico radio, invertire AES-256 senza chiave è computazionalmente infattibile.

### 8.2 Tracciamento nel tempo

**Rischio** Seguire il percorso di un utente nel tempo basandosi su un identificativo fisso o correlabile.

#### Mitigazioni

- per un utente con identificativo  $U_{\text{code}}$  fissato, i valori

$$B_{\text{id}}(S_{\text{slot}}) = \text{AES-ENC}_{K_i}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)})$$

sono diversi per slot diversi  $S_{\text{slot}}$ ;

- senza conoscere  $K_i$  e  $U_{\text{code}}$ , un attaccante che osserva  $B_{\text{id}}(S_{\text{slot}_1})$  e  $B_{\text{id}}(S_{\text{slot}_2})$  non ha un criterio praticabile per stabilire se provengono dallo stesso utente o da utenti diversi;
- la correlazione tra slot diversi è possibile solo nel backend, dove è noto sia  $U_{\text{code}}$  sia la chiave  $K_i$ .

### 8.3 Replay e spoofing di $B_{id}$

**Rischio** Un attaccante ripete (replay) un  $B_{id}$  osservato in passato o in un luogo diverso per simulare la presenza di un altro utente in un istante o in un'area diversa.

#### Mitigazioni

- ogni  $B_{id}$  incorpora uno slot temporale  $S_{slot}^{dec}$ ; al tempo  $t_{unix}$  il backend calcola lo slot corrente

$$S_{slot}^{curr} = \left\lfloor \frac{t_{unix}}{\Delta t_{slot}} \right\rfloor;$$

- la condizione di finestra temporale impone che un  $B_{id}$  sia accettato solo se il tempo corrente  $t_{unix}$  cade nell'intervallo

$$t_{start}^{dec} - \delta \leq t_{unix} \leq t_{end}^{dec} + \delta,$$

dove  $t_{start}^{dec} = S_{slot}^{dec} \cdot \Delta t_{slot}$ ,  $t_{end}^{dec} = (S_{slot}^{dec} + 1) \cdot \Delta t_{slot}$  e  $\delta = \Delta t_{slot}/5$ ;

- un  $B_{id}$  osservato in passato smette di essere risolvibile non appena  $t_{unix} < t_{start}^{dec} - \delta$  oppure  $t_{unix} > t_{end}^{dec} + \delta$ ;
- il replay di  $B_{id}$  fuori dalla sua finestra di validità viene quindi rifiutato dal backend come temporalmente invalido.

### 8.4 Compromissione di chiavi o sistemi backend

**Rischio** Accesso non autorizzato alle chiavi AES  $K_i$  o ai sistemi che le gestiscono e le usano per generare e risolvere i  $B_{id}$ .

#### Mitigazioni

- le chiavi  $K_i \in \{0, 1\}^{256}$  sono generate tramite CSPRNG, indipendenti tra loro e non derivate da una chiave root comune;
- ogni chiave  $K_i$  è valida per un intervallo temporale limitato

$$T_{K_i}, \quad T_{K_i, \text{grazia}} = \frac{T_{K_i}}{5},$$

per un'esposizione complessiva massima  $T_{K_i} + T_{K_i, \text{grazia}}$ ;

- la compromissione di una singola chiave  $K_i$  permette, nel caso peggiore, di risolvere solo i  $B_{id}$  generati con quella chiave e nel relativo intervallo temporale; non consente di recuperare chiavi  $K_j$  con  $j \neq i$ ;
- l'accesso alle chiavi è confinato al backend, e nessuna chiave viene mai distribuita ai client.

### 8.5 Compromissione o furto del dispositivo utente

**Rischio** Un attaccante ottiene accesso al dispositivo dell'utente e può leggere il batch locale di identificativi  $B_{id}$  memorizzato per l'advertising.

#### Mitigazioni

- il batch locale

$$\mathbf{B} = (B_{id,j})_{j=1}^{N_{batch}}$$

copre al più un orizzonte temporale di durata

$$T_{\text{batch}}, \quad N_{\text{batch}} = \frac{T_{\text{batch}}}{\Delta t_{\text{slot}}},$$

quindi l'attaccante può riutilizzare (replay) questi  $B_{\text{id},j}$  solo entro quella finestra temporale;

- i  $B_{\text{id},j}$  sono comunque valori cifrati  $\in \{0,1\}^{128}$ : senza le chiavi  $K_i$ , l'attaccante non può risalire a  $U_{\text{code}}$  né agli slot ( $S_{\text{slot}}^{\text{dec}}$ ) in chiaro;
- le chiavi AES  $K_i$  non risiedono mai sul dispositivo: la compromissione di un singolo telefono non compromette il resto del sistema, né permette di derivare chiavi o identificativi futuri;
- l'impatto della compromissione di un device è quindi limitato alla possibilità di replay dei  $B_{\text{id}}$  contenuti nel batch corrente, per una durata massima pari a  $T_{\text{batch}}$ , ed è considerato accettabile rispetto al beneficio di autonomia del client.

## 9 Alternative considerate e modelli mitigati

### 9.1 Nome in chiaro via BLE

**Idea** Trasmettere direttamente un campo testuale `name` nei pacchetti BLE come identificativo dell'utente.

#### Motivi di scarto

- esporrebbe un dato personale direttamente in chiaro a chiunque nel raggio BLE, cioè

$$\text{name} \longrightarrow \text{persona reale}$$

tramite un mapping banale e stabile;

- renderebbe immediato il tracciamento nel tempo: lo stesso `name` verrebbe osservato in luoghi e istanti diversi, permettendo di costruire una traiettoria temporale per utente;
- soffrirebbe dei limiti di payload BLE per nomi lunghi o caratteri Unicode complessi;
- è in contrasto con il principio di minimizzazione dei dati: il canale radio veicolerebbe più informazione del necessario rispetto all'obiettivo ( $B_{\text{id}}$  opaco è sufficiente).

### 9.2 Identificativo statico non rotante

#### Idea

Assegnare a ogni utente un identificativo fisso

$$U_{\text{static}} \in \{0,1\}^{128}$$

e trasmettere sempre e solo  $U_{\text{static}}$  nei pacchetti BLE.

#### Motivi di scarto

- permetterebbe tracciamento perfetto: per uno stesso utente la sequenza  $(U_{\text{static}}(t_k))_k$  è costante nel tempo, quindi un attaccante può costruire una funzione

$$t \longmapsto \text{posizione}(U_{\text{static}}, t)$$

a partire dagli sniffing;

- anche se  $U_{\text{static}}$  è pseudonimo, è un identificatore permanente e linkabile: una volta correlato a un utente reale, tutte le osservazioni passate e future diventano immediatamente attribuibili;
- non rispetta il requisito di non tracciabilità tra slot temporali.

### 9.3 Identificativi random rotanti con mapping esplicito

#### Idea

Per ogni utente e per ogni slot  $S_{\text{slot}} \in \mathbb{N}$ , il backend genera un token casuale

$$B_{\text{id}}^{\text{rand}} \leftarrow \text{Random}_{128\text{-bit}}() \in \{0, 1\}^{128}$$

e salva una riga in una tabella di mapping esplicita

$$B_{\text{id}}^{\text{rand}} \longrightarrow (U_{\text{code}}, S_{\text{slot}}).$$

Il client trasmette  $B_{\text{id}}^{\text{rand}}$  via BLE; alla risoluzione il backend esegue una lookup su questa tabella.

#### Motivi di scarto

- il mapping esplicito

$$B_{\text{id}}^{\text{rand}} \longrightarrow (U_{\text{code}}, S_{\text{slot}})$$

esiste nel database: se la tabella viene compromessa, un attaccante che ha sniffato  $B_{\text{id}}^{\text{rand}}$  nel tempo può ricostruire in chiaro coppie  $(U_{\text{code}}, S_{\text{slot}})$ ;

- richiede di generare e *persistere* un nuovo token per ogni utente e per ogni slot: lo spazio cresce come

$$O(|\mathcal{U}| \cdot N_{\text{slot}}),$$

dove  $|\mathcal{U}|$  è il numero di utenti e  $N_{\text{slot}}$  il numero di slot coperti dallo storico; ciò implica tabelle molto grandi e job di purge frequenti;

- a parità di comportamento radio (identificativi opachi e rotanti), introduce più stato e complessità rispetto alla soluzione AES corrente, in cui il mapping è *implicito* nel blocco cifrato:

$$B_{\text{id}} = \text{AES-ENC}_{K_i}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)}),$$

e viene ricostruito al volo con una sola AES-DEC.

### 9.4 Modello HMAC-SHA-256 con tabelle di lookup

#### Idea

Per ogni utente si definisce una chiave HMAC

$$k_{\text{HMAC}} \in \{0, 1\}^\ell$$

e per ogni slot  $S_{\text{slot}}$  si calcola un token

$$T = \text{HMAC-SHA-256}(k_{\text{HMAC}}, U_{\text{code}} \parallel S_{\text{slot}}^{(64)}),$$

che viene poi troncato a 16 byte per poter essere inserito nei pacchetti BLE:

$$B_{\text{id}}^{\text{HMAC}} = \text{Trunc}_{128}(T) \in \{0, 1\}^{128}.$$

Il backend pre-calcola, per ogni utente e slot, i token  $B_{\text{id}}^{\text{HMAC}}$  e mantiene una tabella di mapping

$$B_{\text{id}}^{\text{HMAC}} \longrightarrow (U_{\text{code}}, S_{\text{slot}}).$$

#### Vantaggi

- se  $k_{\text{HMAC}}$  viene fornita al client, i  $B_{\text{id}}^{\text{HMAC}}$  possono essere generati direttamente sul dispositivo a partire da  $U_{\text{code}}$  e  $S_{\text{slot}}$ ;



- la risoluzione lato backend diventa una lookup  $O(1)$  su tabella chiave-valore;
- HMAC-SHA-256 è considerato crittograficamente robusto (PRF) con chiave segreta.

### Svantaggi

- richiede tabelle di lookup per slot (o per finestre di slot) da tenere in memoria o su storage molto veloce:

$$O(|\mathcal{U}| \cdot N_{\text{slot}}) \text{ entries};$$

- impone job periodici che generano token per tutti gli utenti a ogni slot, con costo computazionale e I/O significativo al crescere di  $|\mathcal{U}|$ ;
- se  $k_{\text{HMAC}}$  viene distribuita al device, la compromissione del dispositivo espone la chiave per quell'utente, permettendo di generare  $B_{\text{id}}^{\text{HMAC}}$  arbitrari per slot futuri;
- un database che contenga il mapping

$$B_{\text{id}}^{\text{HMAC}} \longrightarrow (U_{\text{code}}, S_{\text{slot}})$$

e venga compromesso insieme a  $k_{\text{HMAC}}$  o ad altri segreti, permette la ricostruzione completa delle identità associate agli ID sniffati.

### Come è stato mitigato

Nella soluzione attuale:

- manteniamo il vantaggio principale del modello HMAC (autonomia nella rotazione lato client), ma usiamo  $B_{\text{id}}$  generati in batch dal backend con AES:

$$B_{\text{id}} = \text{AES-ENC}_{K_i}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)});$$

- non distribuiamo chiavi ai device: il client riceve direttamente i  $B_{\text{id}}$  necessari per un orizzonte  $T_{\text{batch}} = 24 \text{ h}$ , senza conoscere  $K_i$ ;
- evitiamo tabelle di lookup per slot: la risoluzione richiede solo un numero costante di decifrate AES-256 (una per ciascuna chiave candidata in  $\mathcal{K}_{\text{cand}}(t_{\text{unix}})$ ), sempre in  $O(1)$  rispetto al numero di utenti.

## 9.5 Modello AES server-centrico precedente

### Idea originaria

Utilizzare una singola chiave master statica  $K_{\text{enc}} \in \{0, 1\}^{256}$  lato backend per cifrare

$$\text{plaintext\_block} = U_{\text{code}} \parallel S_{\text{slot}}^{(64)}.$$

Per ogni slot  $S_{\text{slot}}$ , il client richiede al backend il corrispondente

$$B_{\text{id}}(S_{\text{slot}}) = \text{AES-ENC}_{K_{\text{enc}}}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)})$$

(e, al più, pochi slot futuri) e lo usa subito per l'advertising BLE. Non sono previsti batch ampi, né rotazioni frequenti di chiave.

### Vantaggi

- logica estremamente semplice lato client: per ogni slot, 1 richiesta  $\rightarrow$  1  $B_{\text{id}}$ ;
- risoluzione sempre in  $O(1)$  con una sola AES-DEC usando  $K_{\text{enc}}$ ;
- nessuna tabella di mapping  $B_{\text{id}} \rightarrow$  utente: il mapping è implicito nella decifrazione.

### Svantaggi

- dipendenza costante dalla rete per aggiornare il  $B_{\text{id}}$ : in assenza di connettività il client non è in grado di avanzare negli slot e non può trasmettere valori validi;
- presenza di una chiave a lunga vita  $K_{\text{enc}}$ : se compromessa, un attaccante può decifrare tutti i  $B_{\text{id}}$  passati e futuri e recuperare  $(U_{\text{code}}, S_{\text{slot}})$  per ogni pacchetto sniffato;
- è difficile limitare il raggio d'impatto temporale di una chiave compromessa: l'esposizione non è limitata a una finestra  $[t_{K_i}^{\text{start}}, t_{K_i}^{\text{grazia}}]$ , ma potenzialmente all'intera storia del sistema.

### Come è stato mitigato

La soluzione attuale può essere vista come una versione “rafforzata” del modello AES originario:

- mantiene la struttura base AES-256:

$$B_{\text{id}} = \text{AES-ENC}_{K_i}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)}),$$

e la risoluzione in  $O(1)$  tramite un numero costante di AES-DEC;

- introduce il batch di durata  $T_{\text{batch}} = 24 \text{ h}$  inviato al client, che riceve una sequenza  $\mathbf{B} = (B_{\text{id},j})_{j=1}^{N_{\text{batch}}}$  e può operare offline finché il batch resta valido;
- sostituisce la singola chiave  $K_{\text{enc}}$  con una sequenza di chiavi master  $(K_i)_{i \in \mathbb{N}}$ , ciascuna valida per un intervallo temporale limitato

$$T_{K_i} = 72 \text{ h}, \quad T_{K_i, \text{grazia}} = \frac{T_{K_i}}{5},$$

riducendo l'impatto di un'eventuale compromissione a una finestra temporale finita;

- usa chiavi  $K_i$  generate in modo indipendente tramite CSPRNG, memorizzate e versionate in modo sicuro nel backend, senza mai distribuirle ai client.

## 10 Sintesi complessiva

In questa sezione riassumiamo in modo compatto il funzionamento complessivo del sistema, le proprietà desiderate e i principali trade-off progettuali.

### 10.1 Architettura in sintesi

- Ogni utente è identificato in modo univoco da

$$U_{\text{code}} \in \{0, 1\}^{64},$$

generato casualmente al momento della registrazione e usato solo nelle comunicazioni sicure tra client e backend.

- Il tempo è discretizzato in slot di durata fissata

$$\Delta t_{\text{slot}} = 600 \text{ s} = 10 \text{ min},$$

con indice di slot

$$S_{\text{slot}} = \left\lfloor \frac{t_{\text{unix}}}{\Delta t_{\text{slot}}} \right\rfloor \in \mathbb{N}.$$

- Il backend mantiene una sequenza di chiavi master AES-256

$$(K_i)_{i \in \mathbb{N}}, \quad K_i \in \{0, 1\}^{256},$$

ciascuna valida per una finestra temporale

$$[t_{K_i}^{\text{start}}, t_{K_i}^{\text{end}}], \quad T_{K_i} = t_{K_i}^{\text{end}} - t_{K_i}^{\text{start}},$$

seguita da un periodo di grazia

$$[t_{K_i}^{\text{end}}, t_{K_i}^{\text{grazia}}], \quad T_{K_i, \text{grazia}} = \frac{T_{K_i}}{5}.$$

La politica di rotazione garantisce che, per ogni istante  $t$ , siano attive al più due chiavi:

$$\left| \left\{ i \in \mathbb{N} \mid t_{K_i}^{\text{start}} \leq t \leq t_{K_i}^{\text{grazia}} \right\} \right| \leq 2.$$

- Ogni identificativo trasmesso via BLE è un blocco AES cifrato:

$$B_{\text{id}} = \text{AES-ENC}_{K_i}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)}) \in \{0, 1\}^{128},$$

dove  $S_{\text{slot}}^{(64)}$  è la rappresentazione a 64 bit dell'indice di slot.

- Per ogni utente, il backend genera periodicamente un batch

$$\mathbf{B} = (B_{\text{id},j})_{j=1}^{N_{\text{batch}}}, \quad N_{\text{batch}} = \frac{T_{\text{batch}}}{\Delta t_{\text{slot}}},$$

associato a un primo slot  $S_1$ , in modo che ogni elemento sia destinato a uno slot:

$$S_j = S_1 + (j - 1), \quad B_{\text{id},j} \longrightarrow S_j.$$

- Il client trasmittente conserva localmente  $\mathbf{B}$  e  $S_1$  e, all'istante  $t$ , calcola lo slot corrente

$$S_{\text{slot}}^{\text{curr}} = \left\lfloor \frac{t_{\text{unix}}}{\Delta t_{\text{slot}}} \right\rfloor$$

per ricavare l'indice

$$j^* = S_{\text{slot}}^{\text{curr}} - S_1 + 1,$$

usando  $B_{\text{id},j^*}$  se e solo se  $1 \leq j^* \leq N_{\text{batch}}$ .

- Il client ricevente effettua solo scanning passivo dei pacchetti BLE, estrae i  $B_{\text{id}}$  relativi al servizio e li invia al backend per la risoluzione, senza possedere né chiavi né mapping locali persistenti.
- Alla risoluzione, il backend:

1. determina l'insieme di chiavi candidate  $\mathcal{K}_{\text{cand}}(t_{\text{unix}})$  (chiave corrente e, se in grazia, chiave precedente);
2. tenta la decifratura di  $B_{\text{id}}$  con ciascuna  $K \in \mathcal{K}_{\text{cand}}$ ;
3. ricava  $(U_{\text{code}}, S_{\text{slot}}^{\text{dec}})$  per le decifrate riuscite e calcola

$$t_{\text{start}}^{\text{dec}} = S_{\text{slot}}^{\text{dec}} \cdot \Delta t_{\text{slot}}, \quad t_{\text{end}}^{\text{dec}} = (S_{\text{slot}}^{\text{dec}} + 1) \cdot \Delta t_{\text{slot}};$$

4. verifica la finestra temporale imponendo che

$$t_{\text{start}}^{\text{dec}} - \delta \leq t_{\text{unix}} \leq t_{\text{end}}^{\text{dec}} + \delta;$$

5. verifica che  $U_{\text{code}}$  corrisponda a un utente valido/attivo;
6. se esiste un'unica soluzione coerente, restituisce i dettagli dell'utente al client ricevente.

## 10.2 Proprietà di sicurezza e privacy

Il disegno complessivo soddisfa i requisiti principali:

- **Minimizzazione sul canale radio:** via BLE transita solo  $B_{\text{id}} \in \{0, 1\}^{128}$ , ovvero un blocco cifrato. Nessun dato personale (nome, email, ecc.) o identificativo persistente viene trasmesso in chiaro.
- **Non tracciabilità a livello radio:** per un utente fisso con  $U_{\text{code}}$  costante, la sequenza

$$(B_{\text{id}}(S_{\text{slot}}))_{S_{\text{slot}}} = (\text{AES-ENC}_{K_i}(U_{\text{code}} \parallel S_{\text{slot}}^{(64)}))_{S_{\text{slot}}}$$

produce valori diversi per slot diversi; un osservatore esterno non può, in pratica, collegare due  $B_{\text{id}}$  a uno stesso utente senza conoscere chiavi e dati interni.

- **Limitazione del replay:** ogni  $B_{\text{id}}$  è accettato solo se, dato lo slot decifrato  $S_{\text{slot}}^{\text{dec}}$  e i corrispondenti

$$t_{\text{start}}^{\text{dec}} = S_{\text{slot}}^{\text{dec}} \cdot \Delta t_{\text{slot}}, \quad t_{\text{end}}^{\text{dec}} = (S_{\text{slot}}^{\text{dec}} + 1) \cdot \Delta t_{\text{slot}},$$

il tempo corrente  $t_{\text{unix}}$  soddisfa

$$t_{\text{start}}^{\text{dec}} - \delta \leq t_{\text{unix}} \leq t_{\text{end}}^{\text{dec}} + \delta.$$

Un replay al di fuori di questa finestra viene rifiutato come temporalmente invalido.

- **Impatto limitato di compromissione di chiave:** ogni chiave  $K_i$  governa un intervallo limitato  $[t_{K_i}^{\text{start}}, t_{K_i}^{\text{grazia}}]$ ; la compromissione di  $K_i$  non permette di derivare né chiavi precedenti né successive, limitando il danno temporale a  $T_{K_i} + T_{K_i, \text{grazia}}$ .
- **Assenza di chiavi lato device:** i client non ricevono mai  $K_i$ ; conservano solo batch di  $B_{\text{id}}$  già cifrati. La compromissione di un device esposto consente al più il replay dei  $B_{\text{id}}$  del batch corrente, ma non la decifratura né la generazione di nuovi valori al di fuori dell'orizzonte  $T_{\text{batch}}$ .
- **Mapping centralizzato e implicito:** il mapping

$$B_{\text{id}} \longrightarrow (U_{\text{code}}, S_{\text{slot}})$$

esiste solo implicitamente tramite la decifratura AES nel backend. Non esistono tabelle persistenti ( $B_{\text{id}} \rightarrow U_{\text{code}}$ ) che, se compromesse, rivelerebbero direttamente le identità.

## 10.3 Scalabilità e complessità

Dal punto di vista computazionale:

- la risoluzione di un singolo  $B_{\text{id}}$  richiede al più un numero costante di operazioni AES-DEC, pari a  $|\mathcal{K}_{\text{cand}}(t_{\text{unix}})| \leq 2$ ; la complessità è quindi  $\Theta(1)$  rispetto al numero totale di utenti;
- non sono necessarie tabelle di lookup proporzionali a  $|\mathcal{U}| \cdot N_{\text{slot}}$ , come negli approcci puramente random o HMAC con pre-calcolo per slot;
- il carico sul backend cresce linearmente con il numero di  $B_{\text{id}}$  da risolvere nel tempo (numero di richieste), ma non con il numero di utenti registrati.

## 10.4 Trade-off e limiti noti

- **Dipendenza dal backend per la risoluzione:** il client ricevente non può risolvere localmente i  $B_{\text{id}}$ ; è richiesta connettività verso il backend per ottenere i dettagli delle persone vicine.

- **Autonomia limitata lato advertising:** il client trasmittente è autonomo finché dispone di un batch valido. Se il batch scade e non è possibile contattare il backend, l'advertising viene sospeso per non emettere  $B_{id}$  non risolvibili.
- **Sincronizzazione temporale:** la correttezza della finestra temporale richiede che l'orologio del dispositivo non derivi eccessivamente nel tempo rispetto al backend; la tolleranza  $\delta$  assorbe drift e latenza, ma non copre derive arbitrarie.
- **Osservatore fisico:** come in tutti i modelli basati su prossimità radio, un osservatore fisico presente nello stesso ambiente può comunque inferire co-presenze (chi è vicino a chi) a livello di contesto reale, anche se non può invertire crittograficamente i  $B_{id}$ .

Nel complesso, la soluzione proposta combina:

- identificativi BLE opachi e rotanti basati su AES-256;
- rotazione periodica delle chiavi master con periodo finito e grace period;
- batch lato client per autonomia offline limitata e controllata;
- risoluzione centralizzata in  $\Theta(1)$  con un numero costante di decifature;
- assenza di mapping espliciti persistenti tra  $B_{id}$  e utenti.

Questi elementi forniscono un compromesso bilanciato tra privacy, non tracciabilità, efficienza BLE, scalabilità backend e semplicità di implementazione lato client.